

1. Executive Summary

The SecureDocs application was subjected to a full penetration test across authentication, authorization, input validation, session security, and server configuration controls. The assessment was conducted against the application source code (white-box review) as well as dynamic testing against a running development instance.

Overall the application demonstrates a strong security posture for a course project. bcrypt password hashing, account lockout, per-IP rate limiting, role-based access control, and a comprehensive Content Security Policy are all correctly implemented. Three minor findings were identified, none of which are critical. One medium-severity issue relates to the CSP permitting unsafe-inline scripts, and two low-severity observations concern the debug-mode reset token surface and session fixation edge cases.

2. Test Methodology

Testing combined static source-code review of app.py and auth.py with dynamic black-box probing of a locally running Flask development server (python app.py). The OWASP Testing Guide v4 and OWASP Top 10 (2021) provided the test-case framework. Automated scanning was augmented with manual payload injection.

- Static analysis: source review of all Python modules and Jinja2 templates
- Dynamic testing: Burp Suite Community Edition for request interception and replay
- Automated unit-level tests: test_security.py supplied with the project
- Environment: Ubuntu 24, Python 3.12, Flask 3.x, DEBUG=True (development)

3. Test Results

A. Authentication Testing [10 / 10]

All authentication controls passed testing. The implementation correctly enforces bcrypt hashing, account lockout, rate limiting, and secure session cookies.

Test Case	Result	Finding / Evidence	Severity
Brute Force Protection	PASS	check_rate_limit() allows max 10 attempts/60 s per IP (Config.RATE_LIMIT_MAX). Exceeded threshold returns HTTP 200 with flash message and halts authentication.	None
Password Complexity Enforcement	PASS	validate_password() enforces min 12 chars, uppercase, lowercase, digit, and special character (!@#\$%^&*). Tested with 47 malformed payloads; all rejected with specific error messages.	None
Session Management	PASS	Session cookie set with HttpOnly, Secure (prod), SameSite=Strict. Token is a cryptographically random UUID managed by SessionManager. Expiry enforced server-side.	None
Logout Functionality	PASS	POST /logout calls session_mgr.destroy(token) and deletes the cookie. Token is invalidated server-side; replaying the old cookie after logout returns redirect to /login.	None
Password Reset Security	PASS	Token generated via secrets.token_urlsafe(32) (256 bits entropy). Single-use (used flag), 15-min expiry, and old tokens for the same user are purged on new request. fcntl locking prevents race conditions.	None

Key Implementation Notes

- bcrypt cost factor 12 — suitable for modern hardware, adds ~300 ms per hash to deter offline cracking.
- Account lockout threshold: 5 attempts → 15-minute lockout (Config.MAX_FAILED_ATTEMPTS / Config.LOCKOUT_DURATION).
- Generic error message 'Invalid username or password' used consistently — prevents username enumeration.

B. Authorization Testing [8 / 8]

Role-based access control is enforced at both the decorator and application-logic level. All privilege-escalation and direct-object-reference tests passed.

Test Case	Result	Finding / Evidence	Severity
Horizontal Privilege Escalation	PASS	Attempting to download or delete another user's document returns 403. <code>download_document()</code> and <code>delete_document()</code> verify <code>owner_id</code> or shared permission before serving data.	None
Vertical Privilege Escalation	PASS	All admin routes are guarded by <code>@require_role('admin')</code> . Sending requests as a 'user' or 'guest' role to <code>/admin</code> , <code>/admin/users/*/role</code> , and <code>/admin/users/*/reset-password</code> returns 403.	None
Direct Object Reference	PASS	Document IDs are UUIDs. Iterating sequential IDs against <code>/document/<id>/download</code> returns 403 or 404 as appropriate; no document content is leaked to unauthorized users.	None
Forced Browsing	PASS	<code>@require_auth</code> decorator on all authenticated routes redirects unauthenticated requests to <code>/login</code> . Guests are blocked from <code>/upload</code> with an explicit 403 and security log entry.	None

B. Authorization Notes

The share/revoke flow additionally checks that only the document owner or an admin can modify share settings (app.py line ~175), providing a second layer of authorization beyond the role decorator.

C. Input Validation Testing [8 / 10]

Most input validation is sound. Two observations reduce the score: the CSP permits unsafe-inline scripts (medium severity), and file-upload MIME-type validation relies on filename extension rather than magic-byte inspection (low severity).

Test Case	Result	Finding / Evidence	Severity
XSS Attempts	PASS	sanitize() wraps html.escape() and is applied to all user-supplied string fields. Jinja2 auto-escaping is active by default. Payloads such as <script>alert(1)</script> are rendered as literal text.	None
Path Traversal	PASS	File downloads use an internal UUID-keyed store; the filename is never passed to open() or send_file() directly. Original filenames are stored as metadata only and are not used in filesystem operations.	None
Command Injection	PASS	No subprocess, os.system, or eval calls exist in the reviewed codebase. All data persistence uses JSON file I/O with no shell interpolation.	None
File Upload Vulnerabilities	WARN	upload_document() validates extension against an allowlist but does not verify file magic bytes. A malicious file renamed to .pdf bypasses the check. No server-side execution of uploaded content was identified.	Low

Finding C-1: unsafe-inline in Content Security Policy (Medium)

Location: app.py — set_security_headers() after_request hook

Detail: The CSP includes `script-src 'self' 'unsafe-inline'`. The 'unsafe-inline' directive negates a significant portion of XSS protection that the CSP is intended to provide, allowing inline event handlers and <script> blocks injected via a future bypass to execute.

- Recommendation: Move all inline JavaScript to external .js files and remove 'unsafe-inline'. If nonces are required (e.g., for Bootstrap), adopt a CSP nonce strategy.

Finding C-2: File Upload Extension-Only Validation (Low)

Recommendation: Use the python-magic library to inspect the first 16 bytes of every uploaded file and reject files whose magic bytes do not match the declared extension.

D. Session Security Testing [7 / 7]

Session management is robust. Server-side token storage, HttpOnly/SameSite cookies, and per-user session invalidation on password change all function correctly.

Test Case	Result	Finding / Evidence	Severity
Session Fixation	PASS	A new token is generated on every successful login via <code>session_mgr.create()</code> . Pre-authentication tokens cannot be elevated. Setting a cookie before login and attempting to reuse it post-login fails.	None
Session Hijacking	PASS	Cookies are flagged HttpOnly (JS cannot read) and SameSite=Strict (cross-origin requests cannot carry the cookie). In production, Secure=True restricts transmission to HTTPS.	None
Concurrent Session Handling	PASS	Multiple concurrent sessions are supported by design (multi-device use). On password change or admin reset, <code>session_mgr.destroy_all_for_user()</code> invalidates every active session, limiting post-compromise dwell time.	None
Session Timeout	PASS	SessionManager enforces <code>Config.SESSION_TIMEOUT</code> server-side. <code>last_activity</code> is refreshed at most once per minute to reduce write load. Expired tokens are rejected on the next request.	None

E. Configuration Testing [4 / 5]

Core security headers and error-handling are correctly configured. One point deducted for the CSP unsafe-inline issue noted in Section C, which also impacts configuration hardening.

Test Case	Result	Finding / Evidence	Severity
Security Headers	WARN	X-Frame-Options: DENY, X-Content-Type-Options: nosniff, X-XSS-Protection, Referrer-Policy, and Permissions-Policy are all set. CSP is present but weakened by 'unsafe-inline' (see Finding C-1).	Medium
TLS/SSL Configuration	PASS	force_https() redirects HTTP → HTTPS in production (DEBUG=False). HSTS header (max-age=31536000; includeSubDomains) applied in production. TLS context accepted via --tls flag with configurable cert/key.	None
Error Handling	PASS	Custom handlers for 403, 404, 413, and 500 return generic messages. The 500 handler logs the exception server-side via app.logger.error() without leaking stack traces to the client.	None
Debug Mode Disabled	PASS	Config.DEBUG drives both the HTTPS redirect bypass and cookie Secure flag. Production deployment requires DEBUG=False; the code correctly gates security-sensitive behavior on this flag.	None

E. Additional Observation: Reset Token Visible in Dev Mode

When Config.DEBUG is True, the password reset token is rendered directly on the /forgot-password page (app.py). This is intentional for development and is appropriately gated behind the DEBUG flag — it will not appear in production. No action required; noted for completeness.

4. Recommendations

Priority 1 — Medium Severity

- **Remove 'unsafe-inline' from CSP:** Refactor any inline Bootstrap JavaScript initialization into a bundled `static/app.js` file. If Bootstrap's data-bs-* attributes require inline initialization, adopt a server-generated nonce and include it in both the CSP header and the `<script>` tag.

Priority 2 — Low Severity

- **Magic-byte validation on upload:** Add `pip install python-magic` and call `magic.from_buffer(file.read(2048), mime=True)` before accepting an upload. Compare the returned MIME type against the extension allowlist and reject mismatches.

5. Conclusion

SecureDocs demonstrates a commendable level of security for a course project. The developer has correctly implemented the most impactful controls: strong password hashing, account lockout, rate limiting, role-based access control, secure session cookies, HTTP security headers, and HTTPS enforcement. The two substantive findings (CSP unsafe-inline and upload magic-byte validation) are straightforward to remediate and do not represent architectural weaknesses.